How ASOF JOIN Is Implemented

Phase 1: rewriting to a LATERAL join

CS631 — ASOF JOIN project

October 29, 2026

1 What the rewrite does

ASOF join is implemented without touching the planner or the executor. The parser recognises the syntax, and the analyser rewrites the query into an equivalent `LATERAL` join that PostgreSQL already knows how to run. Everything downstream is untouched code.

```
-- written by the user
SELECT t.sym, t.ts, q.bid
FROM trades t ASOF LEFT JOIN quotes q
  ON t.sym = q.sym AND t.ts >= q.ts;
```

```
-- what the database actually analyses
SELECT t.sym, t.ts, q.bid
FROM trades t LEFT JOIN LATERAL (
    SELECT * FROM quotes
    WHERE quotes.sym = t.sym AND t.ts >= quotes.ts
    ORDER BY quotes.ts DESC
    LIMIT 1
) q ON true;
```

`LATERAL` is what makes the subquery legal: it allows the subquery to reference `t`, the item to its left in the `FROM` clause. Semantically the subquery is evaluated once per row of `trades`, with that row's values substituted in, and `ORDER BY ... LIMIT 1` picks the single nearest match. `LEFT JOIN ... ON true` supplies the NULL-extension when no match exists; a plain `JOIN` drops the row instead, which is how `ASOF JOIN` and `ASOF LEFT JOIN` differ.

2 Where it happens, and why that choice keeps it small

PostgreSQL processes a query in stages: **parse** (text to a raw tree, no catalog lookups), **analyse** (resolve names and types), **rewrite** (views and rules), **plan**, **execute**.

The rewrite is performed at the *start* of analysis, in `src/backend/parser/parse_clause.c`. `transformFromClauseItem()` walks the `FROM` clause; when it meets a `JoinExpr` with `isAsof` set, it calls `transformAsofToLateralJoin()` and continues with the returned tree:

```
/* ASOF joins are executed as LATERAL joins; rewrite before analysis */
if (j->isAsof)
    j = transformAsofToLateralJoin(pstate, j);
```

The timing matters. Because this runs *before* names are resolved, the function manipulates the raw parse tree — `ColumnRef` nodes holding strings, not `Var` nodes with resolved table and column numbers. It produces a tree identical to what the parser would have built had the user typed the `LATERAL` form by hand. Name resolution, type resolution, `LATERAL` validity checking, lateral-reference marking and planning then all happen afterwards in existing code that has no idea ASOF exists.

That is why Phase 1 is roughly 250 lines rather than several thousand: it is a source-to-source translation, the technique a compiler calls *desugaring*, where a new construct is defined entirely by what it translates into.

3 Building the replacement tree

Finding the inequality

The grammar hands over the whole `ON` clause as one expression. The function flattens its top-level `AND` into a list of conjuncts and looks for exactly one that is an inequality relating the two sides:

- the operator is one of `<`, `<=`, `>`, `>=`;
- one operand is a plain column of the right-hand table;
- the other references columns, but none belonging to the right-hand table — so it can only be evaluated against the left.

Finding two such conjuncts, or none, is an error. The remaining conjuncts — the equality keys and any other filters — are left alone.

Choosing the sort direction

The operator decides which way "nearest" runs, and therefore the `ORDER BY`:

Condition	Meaning	Sort
`t.ts >= q.ts`	latest at or before	`q.ts DESC`
`t.ts > q.ts`	latest strictly before	`q.ts DESC`
`t.ts <= q.ts`	earliest at or after	`q.ts ASC`
`t.ts < q.ts`	earliest strictly after	`q.ts ASC`

Writing the same condition with the operands swapped (`q.ts <= t.ts`) means the same thing, so the code checks both arrangements and flips the direction accordingly.

Assembling it

The replacement is built from ordinary raw-parse-tree nodes:

- a `SelectStmt` whose target list is `*`, whose `FROM` is the original right-hand side, whose `WHERE` is the *entire original `ON` clause*, with the `ORDER BY` above and `LIMIT 1`;
- a `RangeSubselect` wrapping it with `lateral = true`, aliased to the name the right table was already visible under — so references like `q.bid` elsewhere in the query keep resolving;
- a new `JoinExpr` with the original join type, the original left side, that subquery as its right side, and `ON true`.

The input tree is not modified; a new `JoinExpr` is returned, because callers may reuse raw parse trees.

4 The one subtlety: where the equality keys go

The equality keys go *inside* the subquery's `WHERE`, together with the inequality. Putting them in the outer `ON` instead looks equivalent and is not:

```
-- WRONG
trades t LEFT JOIN LATERAL (
    SELECT * FROM quotes WHERE quotes.ts <= t.ts
    ORDER BY quotes.ts DESC LIMIT 1
) q ON t.sym = q.sym
```

That subquery finds the latest quote *of any symbol* before the trade, and the outer condition then rejects it if the symbol happens to differ — returning NULL even though a perfectly good quote for the right symbol exists. Filtering and "take the latest" do not commute: you must restrict the candidates *before* choosing the best one, never after.

This is also why PostgreSQL's optimiser could never perform this rewrite for us. Pushing a predicate below a `LIMIT` changes the result, so the planner correctly refuses to do it; the predicate has to be placed correctly at construction time.

5 What it costs, and what it does not do

The resulting plan is a nested loop with the subquery on the inner side:

```
Nested Loop Left Join
  ->  Seq Scan on trades t
  ->  Limit
        ->  Index Scan Backward using quotes_sym_ts_idx on quotes q
              Index Cond: ((sym = t.sym) AND (ts <= t.ts))
```

With a suitable index this is one index descent per left row: $O(n \log m)$, with random access per probe. Without one it degrades to a sort or scan per left row, which is $O(n \cdot m)$. Phase 1 therefore makes ASOF *expressible*, not fast — it performs exactly as well as the hand-written `LATERAL` query, because it is that query.

Making it fast is Phase 2: sort both inputs by `(key, time)` once and walk them together, keeping a single remembered row, which is $O(n + m)$ — and free of sorting entirely when an index already supplies the order. That plan cannot be expressed as a `LATERAL` join at any price, which is the reason to build a real operator.

Known limitations of the current implementation

- Views store the rewritten `LATERAL` form, so `pg_get_viewdef` does not show ASOF syntax.
- Operands are classified by matching column-reference names textually, since the rewrite runs before name resolution; an unqualified reference is never attributed to the right-hand table.
- The inequality operator is not checked for a usable sort order — which Phase 2 needs.
- The right-hand side must be a table or an aliased subquery.
- The right operand of the inequality must be a bare column, so `ON t.ts >= q.ts + interval '1 min'` is rejected.
- Ties on the match column resolve arbitrarily, as `LIMIT 1` does.